# Reviewer Pack — Noncommutative L^p Norm Lower Bound for Disjointness Communi...

Entry b70c19d6f5c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 13:07:54 UTC

## Noncommutative L^p Norm Lower Bound for Disjointness Communication Matrices

**Entry ID**: `b70c19d6f5c7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 13:07:54 UTC

### 1. Conjecture

**Field A** (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

**Statement**:

> For all n ≥ 1, the Schatten p-norm of the DISJOINTNESS communication matrix M_n satisfies ||M_n||_p ≥ Ω(n^{1/p}) for p ∈ {2, 4, 8}, with equality achieved by the uniform distribution over disjointness instances.

**Rationale (proposer's reasoning)**:

> Noncommutative L^p norms capture operator-level structure of matrices, which may reveal hidden geometric constraints in communication matrices. The conjecture links p-norm scaling to communication complexity via operator space duality, bypassing traditional rank-based methods.

**Taxonomy category**: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `ac30af9f26f968f3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.95 | UNCERTAIN | SAFE |
| NATURAL_PROOFS | SAFE | 0.95 | SAFE | SAFE |

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| ALGEBRIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_disjointness_matrix(n):
    A = [random.randint(0, 1) for _ in range(n)]
    B = [random.randint(0, 1) for _ in range(n)]
    M_n = [[A[i] ^ B[j] for j in range(n)] for i in range(n)]
    return M_n

def matrix_multiplication(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(m)] for i in range(k)]
    return C

def matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    A = [row[:] + [1] for row in matrix]
    U = []
    V = []
    for i in range(min(m, n)):
        max_row = max(range(i, m), key=lambda x: abs(A[x][i]))
        if A[max_row][i] == 0:
            continue
        A[i], A[max_row] = A[max_row], A[i]
        U.append([1 if j == i else 0 for j in range(n)])
        V.append([1 if j == i else 0 for j in range(m)])
        for j in range(i + 1, m):
            factor = A[j][i] / A[i][i]
            A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
    rank = sum(1 for row in U if any(row))
    return rank

def schatten_p_norm(matrix, p):
    singular_values = []
    m, n = len(matrix), len(matrix[0])
    for i in range(min(m, n)):
        max_row = max(range(i, m), key=lambda x: abs(matrix[x][i]))
```

```
        if matrix[max_row][i] == 0:
            continue
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        singular_values.append(abs(matrix[i][i]))
    return sum(singular_value ** p for singular_value in singular_values) ** (1 / p)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_name = "Schatten_p_norm"
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            M_n = generate_disjointness_matrix(n)
            norm_2 = schatten_p_norm(M_n, 2)
            norm_4 = schatten_p_norm(M_n, 4)
            norm_8 = schatten_p_norm(M_n, 8)
            if norm_2 < n ** (1/2) or norm_4 < n ** (1/4) or norm_8 < n ** (1/8):
                conjecture_holds = False
                counterexample = f"n={n}, p=2: {norm_2}, p=4: {norm_4}, p=8: {norm_8}"
            total_metric_value += norm_2 + norm_4 + norm_8
            instances_tested += 3

    return {
        "metric_name": metric_name,
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(r
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif counterexample := [r["counterexample"] for r in results if r["counterexample"]][0]:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[re
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
'n=40, p=2: 6.164414002968976, p=4: 2.4828237961983883, p=8: 1.575697875926216'}
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.3992288909229176, 'instances_tested': 90
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.435291859336928, 'instances_tested': 90,
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.450571445248159, 'instances_tested': 90,
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.4639221878698567, 'instances_tested': 90
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.476655292912502, 'instances_tested': 90,
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.4245870039599913, 'instances_tested': 90
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.4603378015952155, 'instances_tested': 90
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.3885132175919916, 'instances_tested': 90
TRIAL: {'metric_name': 'Schatten_p_norm', 'metric_value': 2.4469539486632144, 'instances_tested': 90
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> parse_fail | next: NONE

## 11. Audit log (LLM calls)

**Total LLM calls**: 10

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 104802 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 109964 |
| 3 | propose | ollama_remote | qwen3:8b | 0 | 0 | 104355 |
| 4 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 23975 |
| 5 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 20550 |
| 6 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 17060 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 18224 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13068 |
| 9 | critic | ollama_remote | qwen3:8b | 0 | 0 | 29685 |
| 10 | judge | ollama_remote | qwen3:8b | 0 | 0 | 21081 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 462763 ms total latency. Provider mix: {'ollama_remote': 10}

*(full prompt+response transcripts available in `research/audit/b70c19d6f5c7.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/b70c19d6f5c7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/b70c19d6f5c7.tar.gz` (if generated)